

Week 6: 资产化数据工厂——编排、回填与可追溯

从“脚本跑通”升级到“数据产品可运营”

Week03 让数据能可靠进来，Week04 让表状态可快照、可回看，Week05 让业务口径可复用。

Week06 要解决的是上线前一定会暴露的问题：

当链路由多个脚本、表、指标和未来索引组成时，团队如何知道哪些资产完成了、哪些分区缺失、失败后该补哪一段、补完后如何验证、证据如何交接？

本周实践对齐 `dataPro-lgtm/omnisupport-copilot` 当前 Week06 实现：`pipelines/data_factory/contracts/run_evidence/week06_run_evidence.schema.json`、`runbooks/week06-data-factory.md` 与 `reports/week06/`。课程页只讲站点内容，不修改项目代码。

本周一句话

把 Week03–Week05 的脚本、manifest、lakehouse baseline、KPI mart、runbook 和 evidence report，组织成一张 可依赖、可分区、可回填、可检查、可追溯、可交接 的资产化数据工厂 v1。

先用一个类比理解 Week06

脚本跑通像厨师说“菜已经做完”。数据工厂要继续回答：这道菜用了哪批原料、哪道工序完成、哪一步质检通过、有没有过期风险、谁批准可以端给客人。

回到 OmniSupport Copilot，“端给客人”不是一个比喻，而是后续工程链路的真实消费：Week07 解析要知道输入资产是否可用，Week08 检索要知道索引基线是否可信，Week10 工具要知道能不能读指标视图，Week11 评测要知道结果来自哪次运行，Week14 治理要能追溯责任边界。

！运行成功不是交付成功

Week06 的交付标准不是“Dagster 页面上有绿色框”，而是另一个人能根据 asset graph、partition、checks、run evidence 和 runbook 判断：这批数据是否可以被 Week07 解析、Week08 检索或 Week11 评测继续消费。

Week06 到底回答什么

问题	Week06 的回答	不属于 Week06 的内容
哪些数据资产完成了？	用 asset graph、materialization 状态和 asset key 说明	不靠口头确认“我跑过了”
失败边界在哪里？	用 daily partition、backfill plan 和 reason code 说明	不盲目全量重跑
结果能不能被下游消费？	用 asset checks、metadata 和 run evidence 说明	不只看 exit code 或日志片段
怎么交给别人？	用 runbook、evidence report 和 delivery summary 说明	不靠作者本人救火

为什么 Week06 重要

- 如果链路只靠“先跑 A、再跑 B”的口头顺序维护，换一个人就无法接手。
- 如果没有 partition boundary，补数只能靠经验，很容易全量重跑或重复写入。
- 如果没有 asset checks，跑完不等于可信，下游仍然可能消费坏数据。
- 如果没有 run evidence，Week07 解析、Week08 检索、Week11 评测和 Week14 治理都缺少可追溯基线。

Week06 不是什么

- 不是把 Dagster 当成好看的 UI 演示。
- 不是重写 Week03 ingest、Week04 lakehouse 或 Week05 dbt 逻辑。
- 不是提前做 Week07 文档解析、Week08 RAG、Week10 工具治理或 Week14 OpenLineage 平台。
- 不是依赖 Dagster+ 专有能能力；Student Core 默认使用 Docker Compose + Dagster OSS + devbox。

小白最容易误解的地方

你可能以为	实际上
Week06 是学 Dagster UI	Week06 是学资产状态、分区、检查和证据，Dagster 只是本周的最小承载工具 ¹
Backfill 就是重跑	Backfill 是对明确 partition window 的受控重算，不是把所有历史数据再跑一遍 ²
有日志就有证据	日志是过程，run evidence 是可交付、可校验、可归档的运行事实
Asset graph 越复杂越专业	Student Core 先做少量关键资产和清晰边界，复杂度以后再加
OpenLineage / lakeFS 现在就要上	本周只预埋 lineage、evidence 和治理字段，完整平台延后 ³

本周在整门课里的位置

上承 Week03

Week03 已经有 ingest、checkpoint、replay/backfill 的脚本思维。Week06 把它升级为 asset-level partition 和 backfill。

接住 Week04

如果 lakehouse 已完成，就记录 snapshot / table state；如果尚未完成，就作为 optional observation，不阻塞主线。

接住 Week05

如果 KPI mart 已完成，就进入 asset graph；如果尚未完成，就用 placeholder / skipped evidence 记录状态。

¹Dagster 的 asset 概念强调“软件定义资产”，适合把脚本输出、表、检查和证据组织成可依赖对象；本周借用的是这个工程模型，而不是把 UI 操作当成学习目标。

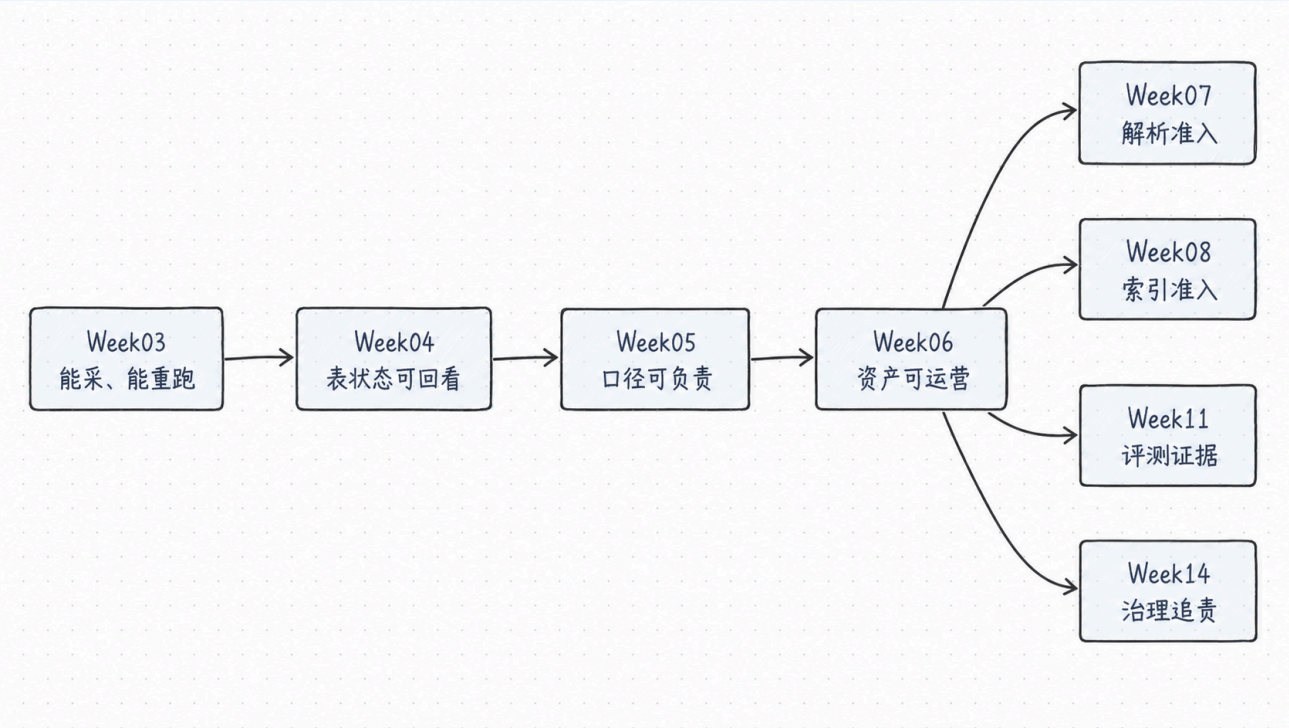
²Dagster 的 partition / backfill 文档把回填和分区边界放在一起讲，本周也采用 daily partition 作为 Student Core 的最小恢复单位。

³OpenLineage 的 object model 围绕 run、job、dataset 组织 lineage 事实；Week06 先让学员理解这些事实为什么需要被记录，完整 OpenLineage 平台后续再展开。

交给 Week07+

Week07 的 parse pipeline、Week08 的 evidence serving、Week11 的 evals 和 Week14 的 governance 都需要 Week06 的 run evidence。

先看 Week06 主线图



这张图只想说明一件事：Week06 不是孤立的编排课。Week03 给脚本和恢复动作，Week04 给表状态记忆，Week05 给指标口径，Week06 把这些组织成资产图和运行证据。后续 Week07/08/11/14 消费的不是“我跑过”，而是“哪份资产、哪个分区、哪类检查、哪份证据可以放行”。

脚本链路 vs 数据工厂

维度	脚本链路	Week06 数据工厂
运行对象	<code>python xxx.py</code> 的执行顺序	可命名、可依赖、可检查的 asset
状态表达	日志里写了 success	asset + partition + checks + evidence 共同表达
失败恢复	让作者凭经验重跑	根据 reason code 选择 retry / replay / backfill / hold
下游交接	口头说明“我跑过了”	runbook + evidence report + delivery summary

任务流 vs 资产流

问题	任务流会怎么说	资产流必须怎么说
今天哪些数据可消费	“任务 B 跑完了”	<code>week06/silver/ticket_fact_partitioned</code> 的 2026-04-17 分区 checks 通过

问题	任务流会怎么说	资产流必须怎么说
上游缺数据怎么办	“重新跑 ingest”	先看 manifest gate, 再生成 backfill dry-run plan
Week04 / Week05 没跑怎么办	“先假设有”	optional observation 写 skipped/not_available, 不阻塞 Student Core
交给下一周用什么	“看这个表”	提供 data release、run evidence、known limitations 和 unblock decision

Student Core / Instructor Scale 边界

层级	Week06 做到什么	不做什么
Student Core	Docker Compose + Dagster OSS + devbox + daily partition + dry-run backfill + 5 类 checks + run evidence	不依赖 Dagster+, 不做完整 OpenLineage 平台, 不要求大规模多租户调度
Instructor Scale	多分区、多批次、更多 external assets、OpenLineage / lakeFS 预告、更多故障演练	不改变 Student Core 的接口, 不把后续周的平台能力提前塞进本周

本周学习路径：先理解，再建图，再补数，再验收，再交接

课时 1 | 先理解

为什么脚本跑通不等于数据产品可运营。先建立问题意识：任务流解决“怎么跑”，资产流解决“谁能消费、为什么可信、出了问题怎么恢复”。

[进入课时 1](#)

课时 2 | 再建图

什么是 asset graph, 哪些东西应该成为 asset。重点不是把所有脚本都搬进 Dagster, 而是给关键数据产品命名、依赖、分组和边界。

[进入课时 2](#)

课时 3 | 再补数

partition / backfill / replay 如何做成受控动作。补数不是“再跑一次”，而是对明确分区、原因、影响范围和幂等边界的恢复动作。

[进入课时 3](#)

课时 4 | 再验收

checks / metadata / run evidence 怎么决定下游放行。跑完只是过程，能不能被 Week07/08/11 消费，要看检查和证据。

[进入课时 4](#)

课时 5 | 再交接

runbook 如何把系统交给另一个人。故障定位、补数、验证、交接都要写成别人能照着执行的操作入口。

进入课时 5

实验 | 资产图 + 回填 + 证据闭环

实验会围绕一个最小闭环展开：

- 启动 Docker Compose / devbox / Dagster
- 加载 Week06 asset graph
- materialize 一个 daily partition
- 制造一个安全的数据缺口
- 执行 backfill / replay dry-run
- 跑 5 类 asset checks
- 生成 `reports/week06/ run evidence`
- 按 runbook 完成交接记录

进入实验

作业 | 提交 Data Factory v1

作业要求提交：

- asset graph summary 或截图
- partition strategy
- 一次精准回填记录
- asset checks 摘要
- run evidence report
- Data Factory Runbook v1
- 已知限制与下一步建议

进入作业

本周交付物如何用在项目里

交付物	项目路径	小白版用途
Data Factory Blueprint	<code>docs/blueprints/week06/week06-data-factory-blueprint.md</code>	写清本周到底做什么、不做什么，防止把 Week06 做成无边界的工具展示
Asset Graph Plan	<code>docs/blueprints/week06/week06-asset-graph.md</code>	说明哪些对象是资产、谁依赖谁、哪些节点只是 optional observation
Partition / Backfill Strategy	<code>docs/blueprints/week06/week06-partition-backfill-strategy.md</code>	说明默认分区、缺口判断、回填窗口和不能全量乱跑的边界
Run Evidence Schema	<code>contracts/run_evidence/week06_run_evidence_schema.json</code>	约束一份运行证据必须包含哪些字段，避免报告变成自由作文
Data Factory Runbook	<code>runbooks/week06-data-factory.md</code>	让另一个人能按步骤观察、物化、补数、检查、记录证据和决定是否放行

交付物	项目路径	小白版用途
Reports	reports/week06/	存放 backfill plan、checks、run evidence 和 delivery summary, 让验收有落点

接下来怎么学

先看课时 1 建立问题意识 → 课时 2 读懂 asset graph → 课时 3 掌握补数边界 → 课时 4 建立验收证据 → 课时 5 写成 runbook → Lab 跑闭环 → Assignment 提交 Data Factory v1

本周关键判断

! Week06 的核心不是“会用 Dagster”

Week06 的核心是：用 asset graph、partition、checks、metadata、evidence 和 runbook, 让数据链路从个人脚本变成团队可运营的数据产品。

延伸阅读

主题	推荐资料	为什么读
Dagster Assets	Dagster Docs: Assets	理解为什么本周把脚本、表和报告组织成软件定义资产
Dagster Partitions and Backfills	Dagster Docs: Partitions and backfills	理解 partition window 为什么是受控补数的边界
Dagster Asset Checks	Dagster API: Asset checks	理解“跑过”为什么还要被检查后才算可交付
Dagster Metadata / Observations	Dagster Docs: Asset materializations and metadata 、 Asset observations	理解 materialization、observation、metadata 如何共同构成运行事实
OpenLineage Object Model	OpenLineage Docs: Object model	理解 Week06 为什么预埋 run、job、dataset 的追溯观念，但不抢跑完整平台
Quarto Mermaid / Callouts	Quarto Mermaid 、 Quarto Callouts	只作为课程站点写法参考，保持整站样式一致

Footnotes